

# InventiGest

Sistema Web de Gestión de Inventario y Ventas

**Avance del Proyecto Integrador — Hito II**  
Implementación funcional de la arquitectura

|                    |                                                                             |
|--------------------|-----------------------------------------------------------------------------|
| <b>Autor</b>       | Ragami Perez                                                                |
| <b>Asignatura</b>  | Arquitectura de Software (CD-IS-1640)                                       |
| <b>Parcial</b>     | II Parcial                                                                  |
| <b>Repositorio</b> | github.com/xddd-hub-star/InventiGest-Arquitectura (rama <code>main</code> ) |

# 1. Resumen del sistema

InventiGest centraliza el control de inventario y ventas de una PyME. Permite registrar productos y existencias, vender descontando el stock de forma automática y transaccional, aplicar descuentos por volumen, restringir funciones según el rol del usuario y consultar un reporte resumen.

En este hito la arquitectura planteada en el Hito I dejó de ser solo un diseño: está implementada en cuatro capas (Presentación, Aplicación, Dominio e Infraestructura) con una API REST en Node.js/Express, una interfaz React, una base de datos relacional PostgreSQL y 23 pruebas automáticas que recorren todas las capas.

| Elemento      | Tecnología                                                                                  |
|---------------|---------------------------------------------------------------------------------------------|
| Frontend      | React 19 + Vite                                                                             |
| Backend       | Node.js + Express 5 (API REST sin estado)                                                   |
| Base de datos | PostgreSQL, acceso con el controlador <code>pg</code> (SQL parametrizado)                   |
| Autenticación | JWT + bcrypt, control de acceso por roles (admin y vendedor)                                |
| Pruebas       | Ejecutor nativo <code>node:test</code> y base PostgreSQL en memoria ( <code>pg-mem</code> ) |

# 2. Correcciones y cambios respecto al I Parcial

El avance del I Parcial (Semana 4) entregó el diseño y la estructura de carpetas de las cuatro capas, con archivos que solo contenían comentarios. Los cambios de este hito son:

| Hito I                                                                                                                        | Hito II                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Estructura de capas con archivos vacíos y un comentario de una línea cada uno.                                                | Las cuatro capas están implementadas y conectadas mediante contratos; ver secciones 3 a 6.                                                                |
| Contratos <code>IProductoRepository</code> , <code>IVentaRepository</code> e <code>INotificacionService</code> sin contenido. | Contratos definidos en el Dominio y se agregan <code>IUsuarioRepository</code> e <code>IAutenticador</code> ; los servicios dependen solo de ellos (DIP). |
| Sin base de datos.                                                                                                            | Base de datos relacional con scripts SQL ( <code>Database/</code> ), claves, restricciones y datos de prueba.                                             |
| Sin pruebas.                                                                                                                  | 23 pruebas automáticas: caso correcto, datos inválidos, reglas de negocio, persistencia y manejo de errores.                                              |
| Tres roles (Administrador, Vendedor, Encargado de Inventario).                                                                | Dos roles: <code>admin</code> y <code>vendedor</code> . Las funciones del Encargado de Inventario quedan a cargo del administrador en este avance.        |

**Observaciones del docente al I Parcial:** pendiente de completar con la retroalimentación recibida en la evaluación del avance anterior, indicando cómo se atendió cada punto.

## Estado de los requisitos definidos en el Hito I

| Código | Requisito                                         | Estado en este avance                                                                           |
|--------|---------------------------------------------------|-------------------------------------------------------------------------------------------------|
| RF01   | Iniciar sesión                                    | Implementado (CU6)                                                                              |
| RF02   | Gestionar usuarios y roles                        | Pendiente: los usuarios se crean con el script de datos de prueba                               |
| RF03   | Registrar y actualizar productos y categorías     | Parcial: registro de productos y ajuste de stock; sin categorías ni edición de precio           |
| RF04   | Consultar catálogo y disponibilidad               | Implementado (CU2)                                                                              |
| RF05   | Registrar venta con descuento automático de stock | Implementado (CU4)                                                                              |
| RF06   | Comprobante de venta                              | Parcial: el detalle de cada venta se puede consultar (CU5); no hay comprobante imprimible       |
| RF07   | Entradas y ajustes de inventario                  | Implementado (CU3)                                                                              |
| RF08   | Alertas de stock mínimo                           | Parcial: se detecta y se notifica por consola; el envío por correo queda para el siguiente hito |
| RF09   | Reportes por periodo                              | Parcial: reporte resumen con filtro de fechas en la API                                         |
| RF10   | Restringir funciones según el rol                 | Implementado                                                                                    |

### 3. Arquitectura actualizada

Las dependencias apuntan hacia el Dominio. Infraestructura implementa los contratos que el Dominio define, de modo que ninguna regla de negocio conoce PostgreSQL, bcrypt ni Express.

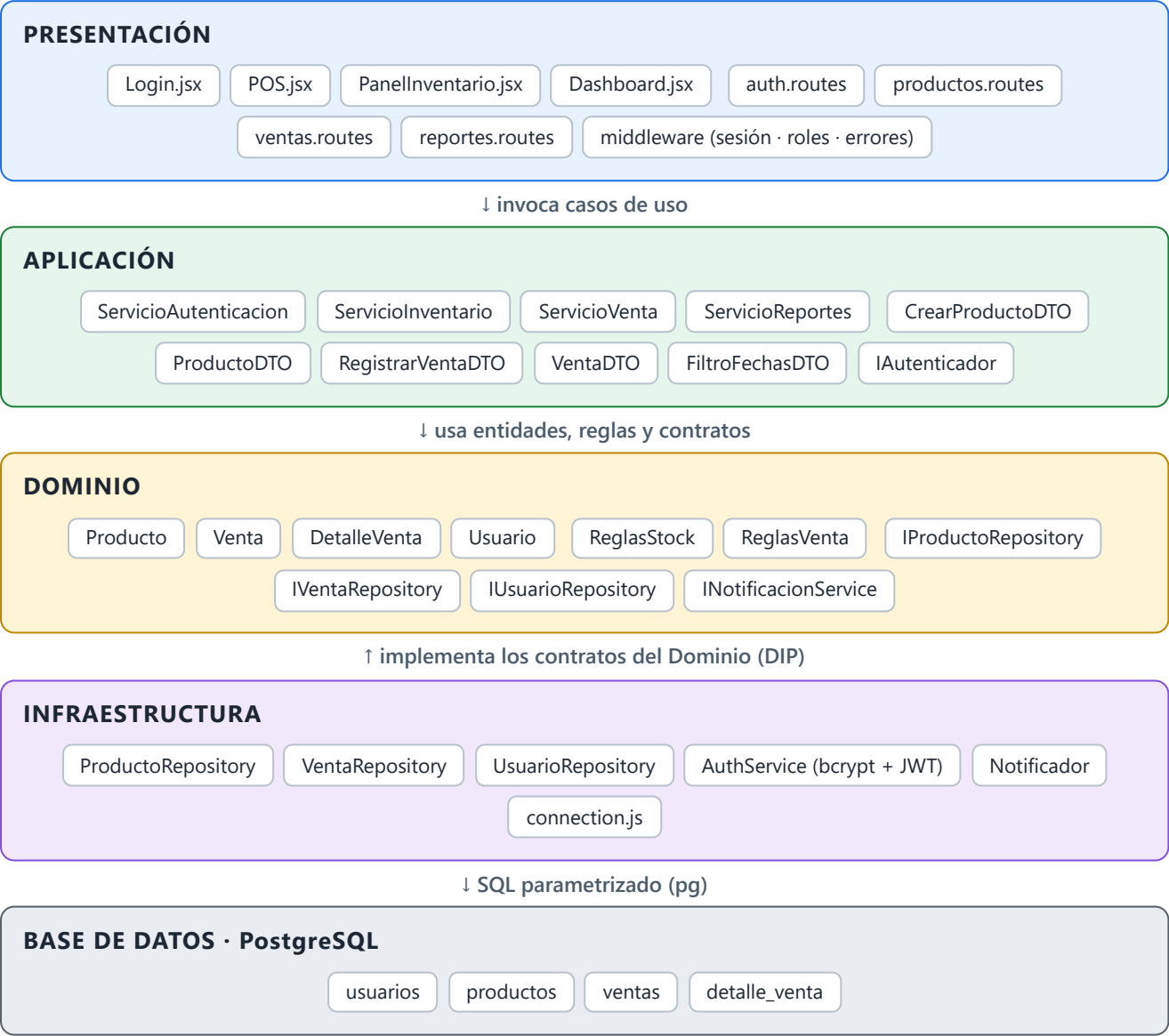


Figura 1. Diagrama de arquitectura en capas con los componentes reales del proyecto.

#### Responsabilidad de cada capa

| Capa         | Componentes reales                                                                  | Responsabilidad                                                                                                                        |
|--------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Presentación | presentation/*.routes.js , middleware.js , app.js ; frontend React                  | Recibir peticiones HTTP, validar la sesión y el rol, traducir errores de dominio a códigos HTTP. No contiene SQL ni reglas de negocio. |
| Aplicación   | ServicioAutenticacion , ServicioInventario , ServicioVenta , ServicioReportes , DTO | Coordinar los casos de uso: cargar datos mediante contratos, invocar al Dominio, persistir y notificar.                                |

| Capa            | Componentes reales                                                                   | Responsabilidad                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Dominio         | Producto , Venta , DetalleVenta , Usuario , ReglasStock , ReglasVenta , contratos    | Contener las reglas importantes del negocio y definir los contratos que necesita de la persistencia. |
| Infraestructura | ProductoRepository , VentaRepository , UsuarioRepository , AuthService , Notificador | Implementar los contratos: acceso a PostgreSQL, hash y tokens, notificaciones.                       |

`compositionRoot.js` es el único lugar donde se conectan las implementaciones con los contratos (inyección de dependencias).

## 4. Reglas de negocio implementadas

| Regla                                                                                                                     | Dónde vive                                                  |
|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| El precio debe ser mayor que cero; el stock y el stock mínimo son enteros no negativos; código y nombre son obligatorios. | Producto (constructor)                                      |
| El stock nunca queda negativo. Una venta o ajuste que lo exceda se rechaza.                                               | ReglasStock.validarDisponibilidad , Producto.descontarStock |
| Una venta debe tener al menos un producto, con cantidades enteras mayores que cero; las líneas repetidas se consolidan.   | ReglasVenta.consolidarItems                                 |
| Descuento por volumen: 5 % si el subtotal llega a 1000 y 10 % si llega a 5000.                                            | ReglasVenta.calcularTotales                                 |
| Si una línea no tiene stock suficiente, la venta completa se rechaza y no se modifica ningún producto.                    | Venta.crear y transacción en VentaRepository.registrar      |
| Stock bajo cuando el stock es menor o igual al mínimo; se emite una alerta.                                               | ReglasStock.esStockBajo , INotificacionService              |
| Solo el administrador registra productos, ajusta stock y consulta reportes.                                               | requerirRol en Presentación                                 |

## 5. Servicios, contratos, interfaces y DTO

| Contrato (interfaz)           | Define                                                                          | Implementación     |
|-------------------------------|---------------------------------------------------------------------------------|--------------------|
| IProductoRepository (Dominio) | obtenerPorId, obtenerPorCodigo, obtenerPorIds, listar, listarStockBajo, guardar | ProductoRepository |
| IVentaRepository (Dominio)    | registrar (transaccional), obtenerPorId, listar                                 | VentaRepository    |
| IUsuarioRepository (Dominio)  | obtenerPorEmail                                                                 | UsuarioRepository  |

| Contrato (interfaz)                            | Define                                                                                  | Implementación                          |
|------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------|
| <code>INotificacionService</code><br>(Dominio) | <code>alertarStockBajo</code>                                                           | <code>Notificador</code>                |
| <code>IAutenticador</code> (Aplicación)        | <code>verificarPassword</code> , <code>emitirToken</code> , <code>verificarToken</code> | <code>AuthService</code> (bcrypt + JWT) |

| Servicio de aplicación             | Casos de uso que coordina                                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>ServicioInventario</code>    | <code>registrarProducto</code> , <code>consultarProductos</code> , <code>obtenerProducto</code> , <code>actualizarStock</code> |
| <code>ServicioVenta</code>         | <code>registrarVenta</code> , <code>consultarVentas</code> , <code>obtenerVenta</code>                                         |
| <code>ServicioAutenticacion</code> | <code>login</code> , <code>verificarToken</code>                                                                               |
| <code>ServicioReportes</code>      | <code>resumen</code>                                                                                                           |

**DTO.** Entrada: `CrearProductoDTO`, `RegistrarVentaDTO`, `FiltroFechasDTO`. Salida: `ProductoDTO`, `VentaDTO`. La API nunca expone las entidades; en particular, `Usuario` mantiene el hash de la contraseña como campo privado y no sale hacia el cliente.

## Orientación a objetos y principios de diseño

- **Encapsulación:** `Producto` guarda su estado en campos privados (`#stock`, `#precio`, ...) y solo cambia mediante `descontarStock` y `ajustarStock`.
- **Abstracción y polimorfismo:** los contratos son clases abstractas; `ProductoRepository` y los demás las extienden y los servicios las usan sin conocer la implementación.
- **SRP:** cada servicio atiende un grupo de casos de uso y las reglas viven en clases propias (`ReglasStock`, `ReglasVenta`).
- **DIP:** los servicios reciben sus dependencias por constructor y dependen de contratos, no de PostgreSQL, bcrypt ni del correo.
- **OCP:** cambiar el notificador (consola por SMTP) o el motor de base de datos se hace agregando una implementación, sin modificar Dominio ni Aplicación.

## 6. Casos de uso implementados

| Caso de uso             | Endpoint                                    | Rol   | Flujo entre capas                                                                             |
|-------------------------|---------------------------------------------|-------|-----------------------------------------------------------------------------------------------|
| CU1 Registrar producto  | <code>POST /api/productos</code>            | admin | Ruta → ServicioInventario → Producto (valida) → ProductoRepository → BD                       |
| CU2 Consultar productos | <code>GET /api/productos , /:id</code>      | todos | Ruta → ServicioInventario → ProductoRepository → ProductoDTO                                  |
| CU3 Actualizar stock    | <code>PATCH /api/productos/:id/stock</code> | admin | Ruta → ServicioInventario → <code>Producto.ajustarStock</code> → repositorio → alerta         |
| CU4 Registrar venta     | <code>POST /api/ventas</code>               | todos | Ruta → ServicioVenta → ReglasVenta + <code>Venta.crear</code> → VentaRepository (transacción) |
| CU5 Consultar ventas    | <code>GET /api/ventas , /:id</code>         | todos | Ruta → ServicioVenta → VentaRepository → VentaDTO                                             |

| Caso de uso        | Endpoint                  | Rol     | Flujo entre capas                                              |
|--------------------|---------------------------|---------|----------------------------------------------------------------|
| CU6 Iniciar sesión | POST /api/auth/login      | público | Ruta → ServicioAutenticacion → UsuarioRepository + AuthService |
| Reporte resumen    | GET /api/reportes/resumen | admin   | Ruta → ServicioReportes → repositorios                         |

Cada caso de uso pasa por todas las capas y termina en la base de datos; ninguno es solo una pantalla. La interfaz React (Login, POS, PanelInventario y Dashboard) consume esta API.

## 7. Diagramas

---

La Figura 1 (sección 3) muestra la arquitectura en capas. A continuación se presentan el diagrama de secuencia y la vista lógica; el modelo de datos está en la sección 8.

## 7.1 Diagrama de secuencia: registrar una venta

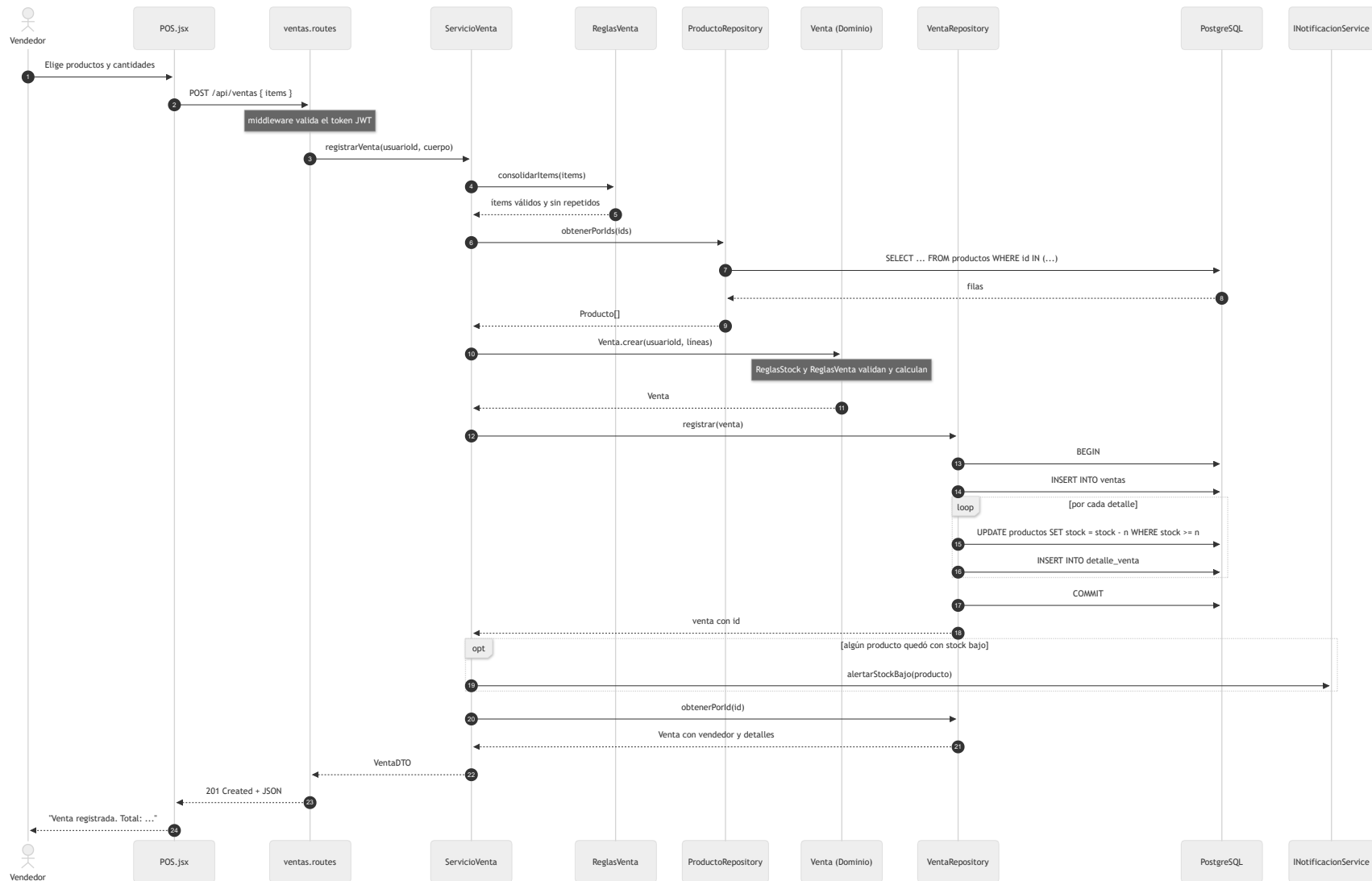


Figura 2. Secuencia del caso de uso más importante, con validación, transacción y alerta de stock bajo.

## 7.2 Vista lógica: módulos, servicios y relaciones

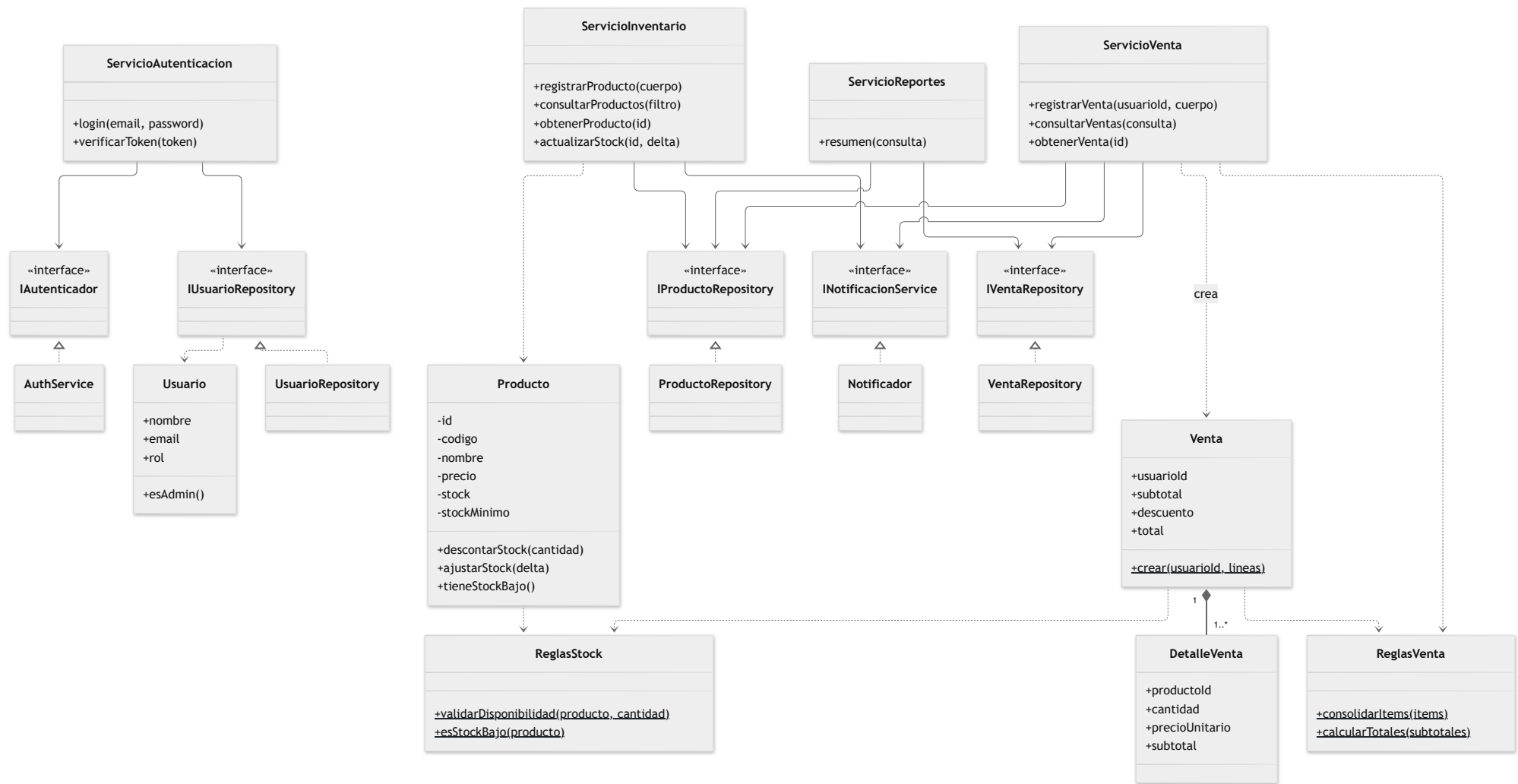


Figura 3. Vista lógica actualizada: servicios de aplicación, entidades y reglas del Dominio, contratos y sus implementaciones.

## 8. Base de datos

Base relacional en PostgreSQL con cuatro tablas, claves primarias y foráneas, restricciones `CHECK` (precio mayor que cero, stock no negativo, cantidades positivas, roles válidos) y restricciones `UNIQUE` (correo y código de producto).

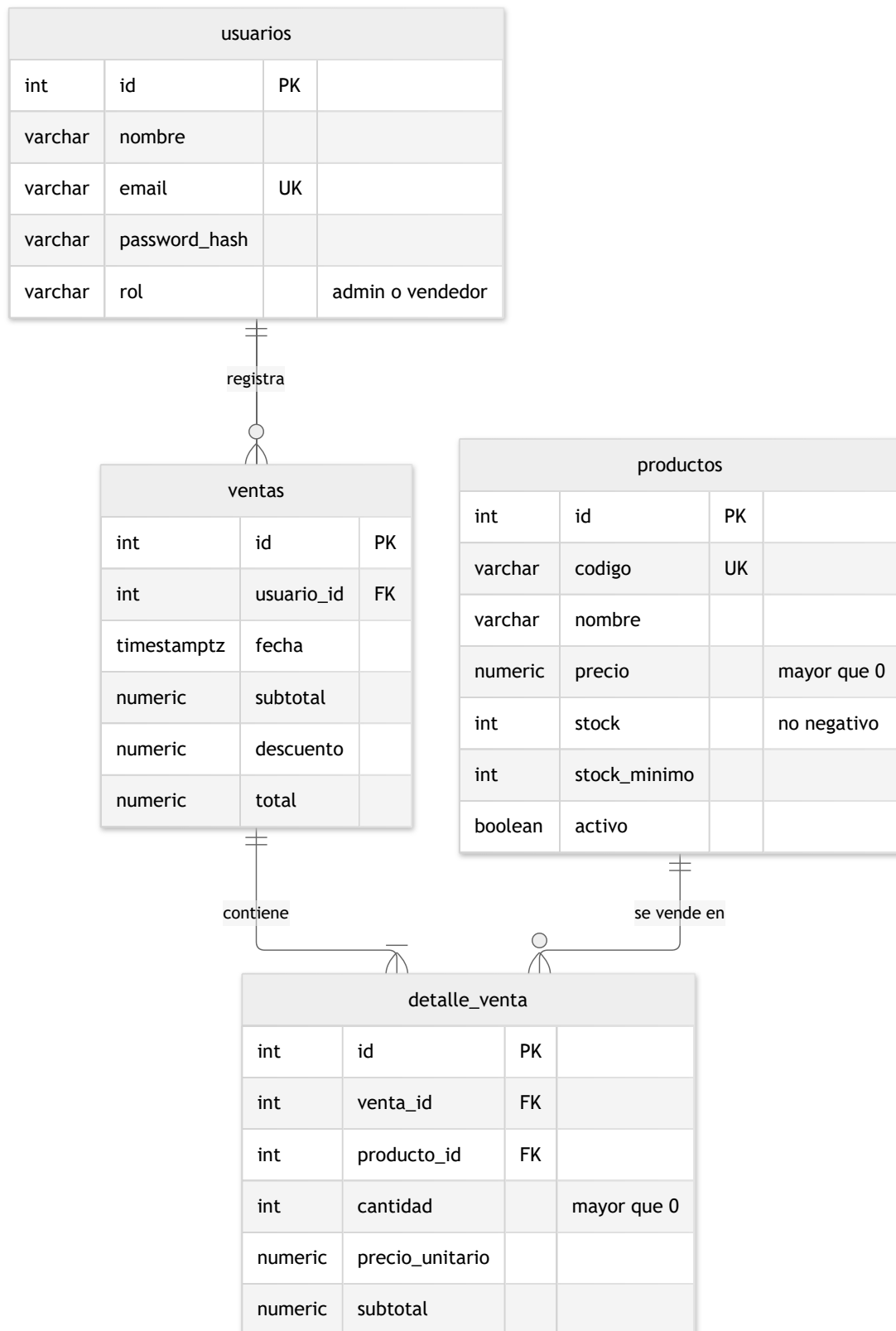


Figura 4. Modelo entidad-relación.

| Script                                 | Contenido                                                          |
|----------------------------------------|--------------------------------------------------------------------|
| Database/01_Crear_Base_Datos.sql       | Crea la base <code>inventigest</code>                              |
| Database/02_Crear_Tablas.sql           | Tablas, claves, restricciones e índices                            |
| Database/03_Datos_Prueba.sql           | 2 usuarios y 5 productos de prueba                                 |
| Database/04_Consultas_Verificacion.sql | Consultas para verificar productos, ventas y coherencia de totales |

El acceso a datos se hace únicamente desde Infraestructura con `pg` y consultas parametrizadas. El descuento de stock en una venta es una actualización condicional ( `WHERE stock >= cantidad` ) dentro de una transacción, por lo que dos ventas simultáneas no pueden dejar el stock negativo.

## 9. Organización del código

```
InventiGest-Arquitectura/
├── backend/
│   ├── src/
│   │   ├── presentation/    app.js · middleware.js · *.routes.js
│   │   ├── application/
│   │   │   ├── services/    ServicioAutenticacion · ServicioInventario · ServicioVenta · ServicioReportes
│   │   │   └── dtos/        CrearProductoDTO · ProductoDTO · RegistrarVentaDTO · VentaDTO ·
│   │   └── FiltroFechasDTO
│   │       ├── interfaces/  IAutenticador
│   │       ├── domain/
│   │       │   ├── entities/ Producto · Venta · Usuario
│   │       │   ├── rules/    ReglasStock · ReglasVenta
│   │       └── interfaces/   IProductoRepository · IVentaRepository · IUserarioRepository ·
│   └── INotificacionService
│       ├── errors.js
│       ├── infrastructure/
│       │   ├── repositories/ ProductoRepository · VentaRepository · UsuarioRepository
│       │   ├── database/     connection.js
│       │   └── services/     AuthService · Notificador
│       ├── compositionRoot.js
│       └── server.js
├── tests/                    dominio.test.js · api.test.js · helpers.js · evidencias.js
├── frontend/src/             App.jsx · api/cliente.js · pages/ (Login, POS, PanelInventario, Dashboard)
├── Database/                 01 a 04 (.sql)
├── documentacion/
└── README.md
```

Verificaciones de dependencias realizadas sobre el código: ningún archivo de `presentation`, `application` o `domain` contiene SQL ni usa el controlador `pg`, y ningún archivo de `domain` importa de otra capa.

## 10. Pruebas, manejo de errores y evidencias de funcionamiento

Se ejecutaron 23 pruebas automáticas: 8 de dominio (entidades y reglas) y 15 de integración que atraviesan Presentación, Aplicación, Dominio, Infraestructura y la base de datos. Resultado de `npm test`:

```
✓ login correcto devuelve token y usuario sin hash de contraseña (1039.9728ms)
✓ login con contraseña incorrecta responde 401 (125.3077ms)
✓ sin token las rutas protegidas responden 401 (5.1875ms)
✓ CU1 registrar producto: caso correcto queda persistido (33.8396ms)
✓ CU1 registrar producto con precio -50 es rechazado (10.4705ms)
✓ CU1 registrar producto con código repetido responde 409 (18.7004ms)
✓ un vendedor no puede registrar productos (403) (5.896ms)
✓ CU2 consultar productos: lista y búsqueda por nombre (29.6627ms)
✓ CU2 consultar un producto inexistente responde 404 (5.5196ms)
✓ CU3 actualizar stock: repone, persiste y rechaza dejarlo negativo (79.9193ms)
✓ CU4 registrar venta: descuento stock, aplica descuento y alerta stock bajo (44.1053ms)
✓ CU4 venta con stock insuficiente es rechazada y no modifica nada (transacción) (61.5463ms)
✓ CU4 venta con datos inválidos responde 400 o 404 (51.5829ms)
✓ CU5 consultar ventas: lista y detalle de una venta (43.8305ms)
✓ reporte resumen: solo admin y refleja ventas y stock bajo (19.6517ms)
✓ Producto rechaza un precio negativo con el mensaje esperado (4.6078ms)
✓ Producto rechaza stock negativo y nombre vacío (0.5733ms)
✓ Producto no permite dejar el stock en negativo (0.5583ms)
```

```
✓ ajustarStock repone y reduce existencias (0.511ms)
✓ tieneStockBajo se activa al llegar al mínimo (0.467ms)
✓ ReglasVenta aplica el descuento por escalón (2.3438ms)
✓ ReglasVenta consolida productos repetidos y valida cantidades (0.9565ms)
✓ Venta.crear descuento stock y calcula totales (0.9533ms)
i tests 23
i pass 23
i fail 0
```

**Alcance de la evidencia.** Las ejecuciones que siguen se realizaron contra la API real, usando una base PostgreSQL en memoria ( `pg-mem` ) creada con los mismos scripts `02_Crear_Tablas.sql` y `03_Datos_Prueba.sql` del proyecto. Se generaron con el script `backend/tests/evidencias.js` .

## Casos ejecutados

### 1. Inicio de sesión correcto

```
POST /api/auth/login
{"email":"admin@inventigest.local","password":"Admin123!"}

HTTP 200
{
  "token": "<jwt>",
  "usuario": {
    "id": 1,
    "nombre": "Ragami Perez",
    "email": "admin@inventigest.local",
    "rol": "admin"
  }
}
```

### 2. Inicio de sesión con contraseña incorrecta

```
POST /api/auth/login
{"email":"admin@inventigest.local","password":"incorrecta"}

HTTP 401
{
  "error": "Correo o contraseña incorrectos."
}
```

### 3. CU1 · Registrar producto (caso correcto)

```
POST /api/productos
{"codigo":"P-100","nombre":"Audífonos","precio":25.5,"stock":12,"stockMinimo":5}

HTTP 201
{
  "id": 6,
  "codigo": "P-100",
  "nombre": "Audífonos",
  "precio": 25.5,
  "stock": 12,
  "stockMinimo": 5,
  "stockBajo": false
}
```

#### 4. CU1 · Registrar producto con precio = -50 (regla de negocio)

```
POST /api/productos
{"codigo":"P-101","nombre":"Producto inválido","precio":-50,"stock":1}

HTTP 400
{
  "error": "El precio debe ser mayor que cero."
}
```

#### 5. CU1 · Registrar producto con código repetido

```
POST /api/productos
{"codigo":"P-001","nombre":"Duplicado","precio":10,"stock":1}

HTTP 409
{
  "error": "Ya existe un producto con el código \"P-001\"."
}
```

#### 6. Control de roles: un vendedor intenta registrar un producto

```
POST /api/productos
{"codigo":"P-102","nombre":"No permitido","precio":10,"stock":1}

HTTP 403
{
  "error": "No tiene permisos para esta operación."
}
```

#### 7. CU2 · Consultar productos (búsqueda "mouse")

```
GET /api/productos?busqueda=mouse

HTTP 200
[
  {
    "id": 2,
    "codigo": "P-002",
    "nombre": "Mouse inalámbrico",
    "precio": 18.5,
    "stock": 35,
    "stockMinimo": 10,
    "stockBajo": false
  }
]
```

#### 8. CU3 · Actualizar stock (+10 al producto 4)

```
PATCH /api/productos/4/stock
{"delta":10}

HTTP 200
{
  "id": 4,
  "codigo": "P-004",
  "nombre": "Cable HDMI 2 m",
  "precio": 6.75,
  "stock": 110,
  "stockMinimo": 20,
  "stockBajo": false
}
```

## 9. CU3 · Actualizar stock dejándolo negativo

```
PATCH /api/productos/4/stock
{"delta":-100000}
```

```
HTTP 409
{
  "error": "Stock insuficiente para \"Cable HDMI 2 m\": disponible 110, solicitado 100000."
}
```

## 10. CU4 · Registrar venta con descuento por volumen (6 monitores)

```
POST /api/ventas
{"items":[{"productoId":3,"cantidad":6}]}
```

```
HTTP 201
{
  "id": 1,
  "fecha": "2026-10-02T04:37:16.091Z",
  "vendedor": "Vendedor Demo",
  "subtotal": 1080,
  "descuento": 54,
  "total": 1026,
  "detalles": [
    {
      "productoId": 3,
      "codigo": "P-003",
      "nombre": "Monitor 24 pulgadas",
      "cantidad": 6,
      "precioUnitario": 180,
      "subtotal": 1080
    }
  ]
}
```

## 11. CU4 · Registrar venta con stock insuficiente (se revierte completa)

```
POST /api/ventas
{"items":[{"productoId":2,"cantidad":1},{"productoId":5,"cantidad":999}]}
```

```
HTTP 409
{
  "error": "Stock insuficiente para \"Memoria USB 64 GB\": disponible 4, solicitado 999."
}
```

## 12. Persistencia: el stock del mouse no cambió tras la venta fallida

```
GET /api/productos/2
```

```
HTTP 200
{
  "id": 2,
  "codigo": "P-002",
  "nombre": "Mouse inalámbrico",
  "precio": 18.5,
  "stock": 35,
  "stockMinimo": 10,
  "stockBajo": false
}
```

### 13. CU5 · Consultar ventas

GET /api/ventas

HTTP 200

```
[
  {
    "id": 1,
    "fecha": "2026-10-02T04:37:16.091Z",
    "vendedor": "Vendedor Demo",
    "subtotal": 1080,
    "descuento": 54,
    "total": 1026,
    "detalles": []
  }
]
```

### 14. Reporte de resumen (admin)

GET /api/reportes/resumen

HTTP 200

```
{
  "cantidadVentas": 1,
  "totalVendido": 1026,
  "totalDescuentos": 54,
  "productosStockBajo": [
    {
      "id": 3,
      "codigo": "P-003",
      "nombre": "Monitor 24 pulgadas",
      "precio": 180,
      "stock": 2,
      "stockMinimo": 3,
      "stockBajo": true
    },
    {
      "id": 5,
      "codigo": "P-005",
      "nombre": "Memoria USB 64 GB",
      "precio": 12,
      "stock": 4,
      "stockMinimo": 5,
      "stockBajo": true
    }
  ]
}
```

Alertas de stock bajo emitidas durante la ejecución: ["P-003"] (el monitor quedó con 2 unidades, por debajo de su mínimo de 3).

## 11. Repositorio Git

| Dato           | Valor                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| URL            | <a href="https://github.com/xddd-hub-star/InventiGest-Arquitectura">https://github.com/xddd-hub-star/InventiGest-Arquitectura</a> |
| Rama principal | main                                                                                                                              |

| Dato                 | Valor                                                       |
|----------------------|-------------------------------------------------------------|
| Contenido            | Código fuente, scripts SQL, README, documentación y pruebas |
| Autor de los commits | Ragami Perez                                                |

## 12. Conclusión del avance

---

El proyecto pasó del diseño a una solución funcional: la arquitectura en cuatro capas está implementada, los seis casos de uso y el reporte atraviesan todas las capas hasta la base de datos, las reglas de negocio residen en el Dominio y las pruebas automáticas validan los casos correctos, los datos inválidos y la reversión transaccional.

### Trabajo pendiente para el siguiente hito

- Gestión de usuarios y roles (RF02), categorías de producto y edición de productos (RF03).
- Comprobante de venta (RF06) y envío real de alertas por correo (RF08).
- Reportes por periodo en la interfaz (RF09), límite de intentos de inicio de sesión y despliegue sobre HTTPS.